# 实验 3 数组与字符串

2025.11.14

## 一. 实验目的

熟练掌握数组的定义、初始化和使用方法；理解数据排序的算法，包括冒泡排序和简单选择排序；掌握数组与循环结构联合解决问题的编程方法；理解字符数组和字符串的联系和区别，熟练掌握使用字符串解决编程问题的方法；熟悉在代码中合理位置加断点，跟踪运行过程的方法，掌握程序调试方法；训练把应用问题转换为程序算法的思维能力。

## 二. 知识要点

### 1. 一维数组的定义、初始化和使用

数组是一组数据类型相同的元素。数组元素按顺序储存在内存中，通过整数下标（或索引）可以访问各元素。在 C 中，数组首元素的下标是 0，所以对于内含 n 个元素的数组，其最后一个元素的下标是 n-1。值得注意的是，编译器和运行的程序都不会检查下标的有效性。

声明一个简单的一维数组形式如下：

type name [ size ];

这里，type 是数组中每个元素的数据类型，name 是数组名，size 是数组元素的个数，C 把数组名解释为该数组首元素的地址。对于传统的 C 数组，要求 size 是整型常量表达式。但是 C99/C11 允许使用整型非常量表达式。这种情况下的数组被称为变长数组，本书不作讨论。

C 把数组看作是派生类型，因为数组是建立在其他类型的基础上。也就是说，无法简单地声明一个数组。在声明数组时必须说明其元素的类型，如 int 类型的数组、float 类型的数组，或其他类型的数组。

为了使程序简洁，常在定义数组的同时给各数组元素赋值，这就是数组的初始化。一维数组的初始化有几种情况：

(1) 在定义数组时对全部数组元素赋予初值。

如：int a[10] = { 0,1,2,3,4,5,6,7,8,9 };

(2) 可以只给数组中的一部分元素赋值。

如：int a[10] = { 0,1,2,3,4 };

(3) 给数组中全部元素赋初值为 0。

如：int a[10] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };

或 int a[10] = { 0 }; //未赋值的部分元素自动设定为 0

(4) 在对全部数组元素赋初值时，由于数据的个数已经确定，因此可以不指定数组长度。编译系统根据初值个数确定数组维数。但不初始化时，必须指定数组长度!

如：int a[5] = { 1,2,3,4,5 };

或 int a[ ] = { 1,2,3,4,5 };

值得注意的是，如果数组长度与提供初值的个数不相同，则方括号中的数组长度不能省略；初始化的时候，初值表不能为空；没有初始化的数组，其元素的值不确定。

数组必须先定义，后使用，而且只能逐个引用数组元素，不能一次引用整个数组。例如，语句 int a[6]; 定义了一个整型数组 a，若要对数组第一个元素赋值为 2，不能直接对 a 赋值，而是 a[0]=2。

定义数组时用到的“数组名[常量表达式]” 和引用数组元素时用的“数组名[下标]”形式相同，但含义不同。定义数组时是指定数组包含的元素个数，引用数组时则是指示某个序号的元素。数组元素与一个简单变量的地位和作用相似，引用时的“下标”可以是整型常量或整型表达式。

**2. 数据排序的基本算法**

常用的排序算法很多，如冒泡排序、直接选择排序、直接插入排序、Shell 排序、快速排序和归并排序等。本书只讨论冒泡排序和直接选择排序。

**(1)冒泡排序**

冒泡排序算法的基本思想是依次比较相邻的两个数据元素，如果反序，则交换它们的位置。对于一个待排序的数据序列，经过一趟交换排序后，具有最大值的数据元素将移到序列的最后位置，值较小的数据元素向最终位置移动一位，这一趟交换过程又称为一趟起泡。对于有 n 个数据元素的数据序列，最多需 n-1 趟排序，第 m 趟对从位置 0 到位置 n-m-1 的数据元素与其后一位的元素进行比较、交换。因此冒泡排序算法可用二重循环实现。

如果用冒泡法对 10 个输入的数排序(由小到大)输出，代码可以如下。

```
#include <iostream>
using namespace std;
#define N 10
int main(void)
{
```

```
    int i, j;
    int list[N], tmp;
    cout<<"Input TEN integers to construct a list:";
    for (i = 0; i < N; i++)
        cin>>list[i];
    cout<<"OK! The list has been constructed:\n";
    for (i = 0; i < N; i++)
        cout<<list[i]<<"    ";
    cout<<"\nTo sort...\n";
    for (i = 0; i < N - 1; i++)
    {
        for (j = 0; j < N - i - 1; j++)
        {
            if (list[j] > list[j + 1])
            {
                tmp = list[j];
                list[j] = list[j + 1];
                list[j + 1] = tmp;
            }
        }
    }
    cout<<"Finished! The list has been sorted:\n";
    for (i = 0; i < N; i++)
        cout<<list[i]<<"    ";
    return 0;
}
```

**(2)直接选择排序**

直接选择排序的基本思想是依次选择出待排序数据中的最小者将其有序排列，具体过程是：首先通过 n-1 次比较，从 n 个数中找出最小的， 将它与第一个数交换，这是第一趟选择排序，结果最小的数被安置在第一个元素位置上；再通过 n-2 次比较，从剩余的 n-1 个数中找出关键字次小的记录，将它与第二个数交换，这是第二趟选择排序；重复上述过程，共经过 n-1 趟排序后，排序结束。

用简单选择法对 10 个数排序的参考代码如下。

```
#include <iostream>
using namespace std;
```

```
#define N 10
int main(void)
{
    int i, j, min;
    int list[N], tmp;
    cout<<"Input TEN integers to construct a list:";
    for (i = 0; i < N; i++)
        cin>>list[i];
    cout<<"OK! The list has been constructed:\n";
    for (i = 0; i < N; i++)
        cout<<list[i]<<"    ";
    cout<<"\nTo sort...\n";
    for (i = 0; i < N - 1; i++)
    {
        min = i;
        for (j = i + 1; j < N; j++)
        {
            if (list[min] > list[j])
                min = j;
        }
        if (min != i)
        {
            tmp = list[i];
            list[i] = list[min];
            list[min] = tmp;
        }
    }
    cout<<"Finished! The list has been sorted:\n";
    for (i = 0; i < N; i++)
        cout<<list[i]<<"    ";
    return 0;
}
```

**3. 二维数组的定义、初始化和使用**

声明一个简单的二维数组形式如下：

```
type name [ size1][size2];
```

可以把二维数组看作是一种特殊的一维数组,它的元素又是一个一维数组。例如, 有语句 float a[3][4]; 则可以把 a 看作是一个一维数组, 它有 3 个元素: a[0], a[1], a[2], 每个元素又是一个包含 4 个元素的一维数组。

用矩阵形式（如 3 行 4 列形式）表示二维数组，是逻辑上的概念，能形象地表示出行列关系。而在内存中，各元素是连续存放的，不是二维的，是线性的。C 语言中，二维数组中元素排列的顺序是按行存放的。

多维数组的初始化有四种方式:

(1)给全部元素赋初值

如：int a[2][3]={{10,11,12},{13,14,15}};

或 int a[2][3]={10,11,12,13,14,15};

(2)给部分元素赋初值

如：int a[2][3]={{10,11},{13}};

或 int a[2][3]={10,11,13};

此时，没有明确初始化的元素被自动初始化。

(3)给全部元素赋初值时，不指定第一维的长度，但要指定第二维的长度

如：int a[ ][3]={{10,11,12},{13,14,15}};

或 int a[ ][3]={10,11,12,13,14,15};

(4)给部分元素赋初值时，不指定第一维的长度，但要指定第二维的长度

如：int a[ ][3]={{10,11},{13}};

使用二维数组元素时，同样要注意不能超出其定义的范围。如有语句 int a[2][3]; 则 a[2][3]=5 就是超出定义范围的引用。

**4. 字符数组和字符串**

字符数组是存放字符数据的数组。字符型数据是以整数形式(ASCII 代码)存放的，因此也可以用整型数组来存放字符数据，但浪费存储空间。

C 语言没有专门用于储存字符串的变量类型，字符串都被储存在 char 类型的数组中。数组由连续的存储单元组成，字符串中的字符被储存在相邻的存储单元中，每个单元储存一个字符。数组末尾位置的字符\0，这是空字符（null character)，C 语言用它标记字符串的结束。空字符不是数字 0，它是非打印字符，其 ASCII 码值是（或等价于）0。C 中的字符串一定以空字符结束，这意味着数组的容量必须至少比待存储字符串中的字符数多 1。

一维字符数组存放一个字符串，二维字符数组存放多个字符串，行数是字符串的个数。

如果在定义字符数组时不进行初始化，则数组中各元素的值是不可预料的。如果花括号中提供的初值个数（即字符个数）大于数组长度，则出现语法错误。如果初值个数小于数组长度，则只将这些字符赋给数组中前面那些元素，其余的

元素自动定为空字符(即′\0′)。如果提供的初值个数与预定的数组长度相同，在定义时可以省略数组长度，系统会自动根据初值个数确定数组长度。

字符数组的初始化可以像普通数组一样初始化，也可以用字符串常量给字符数组赋初值。

如：char a[6] = {“China”};

或 char a[6] = “China”;

也可以不指定数组长度，如：

char a[ ] = “China”;

或 char a[ ] = {“China”};

此时的数组长度是 6，字符串长度是 5。

**字符串的输入：**

① 每次读取一行字符串输入

stream 中的类（如 cin）提供了一些面向行的类成员函数：getline( )读取一行输入，直到到达换行符。

**getline( )函数**通过回车键输入的换行符来确定输入结尾。可以使用 cin.getline( )调用。该函数有两个参数。第一个参数是用来存储输入行的数组的名称，第二个参数是要读取的字符数。如果这个参数为 20，则函数最多读取 19 个字符，余下的空间用于存储自动在结尾处添加的空字符。getline( )成员函数在读取指定数目的字符或遇到换行符时停止读取。

例如，假设要使用 getline( )将姓名读入到一个包含 20 个元素的 name 数组中。可以使用这样的函数 cin.getline(name, 20 )调用。

② 混合输入字符串和数字

混合输入数字和面向行的字符串会导致问题。下面代码想实现姓名、年龄和 ID 的输入。

```
char name[20];
int year;
char ID[20];
cin.getline(name, 20);
cin >> year;
cin.getline(ID, 20);
cout << name << year << ID;
```

但是实际输入输出如下，ID 的内容并没有机会输入。

```
tony
23
```

tony23

可以在 cin >> year;后增加 cin.get();语句，

按上述方法修改后可以正常工作了，运行结果如下：

tony

23

1231

tony231231

## 三. 示例讲解

**示例 1：数学黑洞。任给一个 4 位正整数，其各位上的数字不全相同，将数字重新组合成一个最大的数与最小的数相减，重复这个过程，最多 7 步，必得 6174。对任给的 4 位正整数(各位上的数字不全相同)，编程输出掉进黑洞的步数。例如：输入：1234，则输出：3。**

这是实验 2 中的一个题目，可以用控制结构来实现，也可以用数组和排序来实现更简洁。下面的代码就是用数组和冒泡排序实现的。

```
#include <iostream>
using namespace std;
int main()
{
    int data;
    cout<<"Please input a data with four digits\n";
    cin>>data;
    int temp = data, count = 0, digit[4] = {0}, i, j, max, min;
    while (temp != 6174)
    {
        digit[3] = temp % 10;
        temp /= 10;
        digit[2] = temp % 10;
        temp /= 10;
        digit[1] = temp % 10;
```

```
        temp /= 10;
        digit[0] = temp % 10;
        cout<<"第"<<count+1<<"次处理前的
"<<digit[0]<<digit[1]<<digit[2]<<digit[3]<<"\n";
        for (i = 0; i < 3; i++) //  排序
        {
            for (j = 0; j < 4 - i - 1; j++)
            {
                if (digit[j] > digit[j + 1])
                {
                    temp = digit[j];
                    digit[j] = digit[j + 1];
                    digit[j + 1] = temp;
                }
            }
        }
        max = digit[0] + digit[1] * 10 + digit[2] * 100 + digit[3] * 1000;
        min = digit[3] + digit[2] * 10 + digit[1] * 100 + digit[0] * 1000;
        cout<<"最大值：  "<<max<<",最小值：  "<<min<<"\n";
        temp = max - min;
        count++;
    }
    cout<<"共执行了"<<count<<"次\n";
    return 0;
}
```

**示例 2：编写程序实现，由键盘读入一个字符串，其内有若干个不同的字符，要求程序将字符串中所有非数字删除后输出。**

```
#include <iostream>
using namespace std;
```

```
int main(void) {
    char str[81], str2[81];
    int p,q;
    cin>>str;
    for(p=0,q=0; str[p]!='\0'; p++)
            if( str[p]>='0' && str[p]<='9' )
                str2[q++]=str[p];
    str2[q]='\0';
    cout<<str2;
}
```

**示例 2：已知一个二维数组，找出其最大元素值及其行列号。**

因为是二维数组，可以用两层嵌套的 for 循环来遍历数组元素。寻找最大值和行列号，可以定义两个整型变量记录行列号，初始化为第一个元素的行列号，最大值可以用行列号引用数组实现，也可以定义一个变量记录，示例代码用了后者。循环遍历数组，保持行列号指向当前最大值。

```
#include <iostream>
using namespace std;
int main(void)
{
    int a[3][4] = {{1, 2, 3, 4}, {9, 8, 7, 6}, {-10, 10, -5, 2}};
    int i, j, row = 0, colum = 0, max;
    max = a[0][0];
    for (i = 0; i <= 2; i++)
        for (j = 0; j <= 3; j++)
            if (a[i][j] > max)
            {
                max = a[i][j];
                row = i;
                colum = j;
```

```
            }
        cout<<"max="<<max<<",row="<<row<<", colum="<<colum<<"\n";
        return 0;
    }
```

**四．实验内容（需要将程序与运行结果同屏截图，放在邮件里发送本次上机作业，共 9 张图）**

**1.** 编程实现一个**成绩分析系统**。用户输入 10 个学生的成绩(0-100)，请输出 90 分以上学生的百分比和低于平均分的学生的百分比。

**2.** 编程实现一个金秋舞蹈大赛**打分系统**，完成下列功能：

(1) 十个评委输入分数（80-100），当某位评委输入的分数不合理时，提示该编号评委重新输入。如：最终输入 80 90 85 85 80 90 95 96 97 99

(2) 十个合理分数输入后，利用冒泡法将 10 个分数从大到小排序输出。如：输出 99 97 96 95 90 90 85 85 80 80

(3) 增加了第 11 位评委打分，打分后将该分数插入到已排序的分数中，从大到小输出 11 个分数。如：增加 94 分， 则输出：99 97 96 95 94 90 90 85 85 80 80

(4) 计算 11 个评委的平均分并输出。

提示：每完成一个小功能请输出结果，来测试是否正确，正确后再开发下一个功能。

**3. 编写一个简单的数独程序。**定义一个 3 行 3 列的二维数组；当满足每一行、每一列内的数字均含 1, 2, 3 时，表明它能构成一个简单数独。键盘读入用户输入的 3 行 3 列二维数组，判断其是否为正确的简单数独。

**4. 线性代数:(。**为一个 2 行 2 列的矩阵定义一个二维数组，设计并实现该矩阵的逆矩阵求解并输出。

**5.** 由键盘读入一个字符串，内有若干个字符，将字符串中所有的非数字和非字母删除，只保留数字和字母，并输出修改后的字符串。

6. **加密**。方法为：对八位数每一位上的数字，将其加 5 再对 10 求余后作为新的数字，然后将其右移 3 位，再倒序，例如 12345678 → 67890123 → 12367890 → 09876321。编程对用户输入的八位数加密输出结果。

7. **信号滤波器。**均值滤波器和中值滤波器是信号处理中十分常用的噪声去除方法，请阅读下面的说明后完成平滑滤波和中值滤波后输出各自的结果数据。

有一组数据{1.8, 1.5, 1.1, 0.5, 0.9, 1.1, 1.4, 1.8, 2.3, 2.8, 2.4, 2.1, 1.6, 1.4, 1.1}，请对该数据进行如下的滤波操作。(滤波后的结果要放在新数组里，想想为什么？)

* 窗口宽度为 3 的均值平滑滤波：$x_i' = \frac{x_{i-1} + x_i + x_{i+1}}{3}$。两端的数据可以保持不变。

* 窗口宽度为 3 的中值滤波：$x_i' = median(x_{i-1}, x_i, x_{i+1})$，median 表示三个值的中间值（不是最大、也不是最小值）。两端的数据可以保持不变。

8. **LBP 图像特征。**LBP（局部二值模式）特征是模式识别中常用的图像特征，可以用来进行人脸识别，图像分类等。LBP 特征指的是以矩阵中心元素为阈值，相邻的 8 个元素与中心元素值进行比较，若周围元素大于中心元素值，则该元素点的位置被标记为 1，否则为 0。 然后将这 8 个数从首元素开始顺时针依次排列形成一个二进制数字，再将这个二进制数字转换为十进制数，就是中心像素的 LBP 值。

如：矩阵 $\begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{matrix}$ 首先转换为 $\begin{matrix} 0 & 0 & 0 \\ 0 & 5 & 1 \\ 1 & 1 & 1 \end{matrix}$ 得到二进制数字 00011110，转换为十进制数即 30。

编程要求: 编写主函数实现 3X3 矩阵的输入并计算 LBP 特征后输出得到的结果。

9. **字符串解读。**如果给你一个字符串 s ，它由数字（'0'-'9'）和'#'组成。我们希望按下述规则将 s 映射为一些小写英文字符：字符（'a'-'i'）分别用（'1'-'9'） 表示。字符（'j'-'z'）分别用（'10#'-'26#'）表示。

返回映射之后形成的新字符串。

例如：输入： s = "10#11#12"，则输出： "jkab"；

输入： s ="12345678910#11#12#13#14#15#16#17#18#19#20#21#22#23#24#"

输出： "abcdefghijklmnopqrstuvwx"

**10.**（选做）**双色球彩票投注**生成器：双色球的规则是六个红球（1~33 号，不重复）加一个蓝球（1~15 号），全部被抽中则中一等奖（没有顺序要求）。编程实现以下功能：

每次运行随机得到一注号码（六红 + 一蓝）作为抽奖结果，不显示在控制台；

提示选择操作：

① 手动输入一注自选号码，然后根据上面的结果判断是否中奖，并输出奖项（六红一蓝全部相同，中一等奖；六红相同，蓝色不同为二等奖；五红一蓝相同为三等奖；五红相同或四红一蓝相同为四等奖；四红或三红一蓝相同为五等奖；其他只要有一个蓝色相同即为六等奖）；

② 随机产生投注号码并判断是否为一等奖，如果不是则一直随机产生直到一等奖为止，输出一共产生了多少注最终才获得了一等奖。（不要沉迷彩票投注～）